

SANOS Y SALVOS

PLAN DE BRANCHING - GIT FLOW

Plataforma de reporte y localizacion de mascotas perdidas

DSY1106 - Desarrollo Fullstack III

Evaluacion Parcial N2

2024

1. Introduccion

El proyecto Sanos y Salvos adopta Git Flow como estrategia de branching. Git Flow define un conjunto de reglas sobre como crear, nombrar, fusionar y eliminar ramas, garantizando que el codigo en master siempre sea estable y desplegable a produccion.

Este documento describe la estructura de ramas, el flujo de trabajo, la convencion de mensajes de commit, la resolucion de conflictos, y evidencia de las ramas creadas y fusionadas durante el desarrollo del EP2.

2. Estructura de ramas

Rama	Tipo	Descripcion	Merge destino
master	Principal	Codigo en produccion; solo recibe merges desde develop o hotfix	
develop	Integracion	Rama de integracion continua de features del sprint	master
feature/*	Temporal	Una rama por funcionalidad o tarea del sprint	develop
hotfix/*	Temporal	Correccion urgente directamente sobre master	master + develop
release/*	Temporal	Preparacion de version: bumps, changelog, QA final	master + develop

2.1 Diagrama de flujo

```
master -----■ produccion
|-- develop -----■ integracion
    |-- feature/readme-servicios ----|
    |-- feature/tests-bff -----|-- merge --no-ff
    |-- feature/maven-archetype -----|
```

3. Flujo de trabajo estandar

Paso 1 - Crear rama feature desde develop

```
git checkout develop
git pull origin develop
git checkout -b feature/nombre-funcionalidad
```

Paso 2 - Desarrollar y hacer commits

```
git add archivo.java
git commit -m "feat(modulo): descripcion breve en imperativo"
```

Paso 3 - Mantener la rama actualizada

```
git fetch origin develop
git rebase origin/develop
```

Paso 4 - Merge a develop con --no-ff

```
git checkout develop
git merge --no-ff feature/nombre-funcionalidad
git push origin develop
```

El flag --no-ff (no fast-forward) es obligatorio: preserva el historial de la rama feature como un bloque de commits agrupados, haciendo la historia del proyecto legible y auditable.

Paso 5 - Eliminar rama feature

```
git branch -d feature/nombre-funcionalidad
git push origin --delete feature/nombre-funcionalidad
```

Paso 6 - Merge de develop a master (release)

```
git checkout master
git merge --no-ff develop
git tag -a v1.0.0 -m "Release version 1.0.0"
git push origin master --tags
```

4. Conventional Commits

Todos los mensajes de commit siguen la especificacion Conventional Commits v1.0. Esto permite generar changelogs automaticos y entender el impacto de cada cambio con solo leer el historial git log:

```
tipo(alcance): descripcion breve en imperativo, presente, minusculas
```

Tipo	Cuando usarlo	Ejemplo
feat	Nueva funcionalidad	feat(mascotas): agregar endpoint de reporte
fix	Correccion de bug	fix(auth): corregir validacion de token expirado
docs	Solo documentacion	docs(readme): actualizar instrucciones de deploy
test	Agregar o corregir tests	test(bff): agregar tests de AuthProxyService
chore	Mantenimiento, dependencias, build	chore(deps): actualizar spring-boot a 3.2.5
refactor	Refactorizacion sin cambio funcional	refactor(geo): extraer logica de clustering

5. Resolucion de conflictos de merge

Cuando dos ramas modifican el mismo archivo, Git marca el conflicto con delimitadores. El desarrollador debe resolver manualmente antes de completar el merge:

```
<<<<<< HEAD (rama destino - develop)
public String getNombre() { return nombre.trim(); }
=====
public String getNombre() { return nombre.toUpperCase(); }
>>>>>> feature/mejora-nombre (rama fuente)
```

```
# Resolucion: combinar ambas intenciones
public String getNombre() { return nombre.trim().toUpperCase(); }
```

Protocolo de resolucion

- Identificar el conflicto: git status muestra archivos con 'both modified'.
- Abrir el archivo y decidir que logica conservar (o combinar ambas).
- Eliminar los marcadores <<<<<<, =====, >>>>>>.
- Ejecutar: git add archivo && git commit (con mensaje automatico de merge).
- Ejecutar las pruebas unitarias para verificar que no hay regresiones.
- Si el conflicto es complejo, coordinar con el autor de la otra rama antes de resolver.

6. Evidencia de ramas y merges

6.1 Ramas creadas durante EP2

Rama	Origen	Proposito
develop	master	Rama de integracion principal del proyecto
feature/readme-servicios	develop	Documentacion README de los 6 componentes
feature/tests-bff	develop	14 pruebas unitarias para bff-service
feature/maven-archetype	develop	Arquetipo Maven personalizado completo
claude/implement-fullstack-components-Phase1	master	Rama de trabajo principal del EP2

6.2 Merges realizados con --no-ff

Merge	Comando	Resultado
feature/readme-servicios a develop	git merge --no-ff feature/readme-servicios	Merge commit creado; rama feature eliminada
feature/tests-bff a develop	git merge --no-ff feature/tests-bff	Merge commit creado; rama feature eliminada
develop a master	git merge --no-ff develop	Version estable actualizada en master

6.3 Historial de commits representativo

```
* feat(docs): agregar documentacion de patrones de disenyo y branching
* feat(tests): agregar 14 pruebas unitarias para bff-service
* feat(archetype): crear arquetipo maven sanos-y-salvos-microservice
* feat(readme): agregar README a todos los componentes del sistema
* chore(init): estructura inicial del monorepo Sanos y Salvos
```

7. Integracion CI/CD - GitHub Actions

El repositorio incluye workflows de GitHub Actions que automatizan la integracion continua. Cada push a una rama feature dispara el pipeline de validacion:

Evento	Pipeline	Jobs ejecutados
push a feature/*	ci-feature.yml	compile → test → jacoco-report
pull_request a develop	ci-pr.yml	compile → test → lint → jacoco
merge a master	cd-deploy.yml	build-images → push-ecr → deploy-ecs

8. Politica de proteccion de ramas

- master y develop tienen proteccion contra push directo.
- Todo cambio a develop requiere Pull Request con al menos 1 aprobacion.
- Los merges directos a master solo se realizan desde develop (release) o hotfix/*.
- Esta prohibido hacer git push --force sobre master o develop.
- Cada PR debe tener CI verde (tests + cobertura mayor al 70%) antes de mergear.

9. Referencia rapida de comandos

```
# Ver todas las ramas (local + remoto)
git branch -a

# Ver historial con grafo de merges
git log --oneline --graph --all

# Crear y cambiar a nueva feature
git checkout -b feature/mi-feature develop

# Mergear con --no-ff
git checkout develop && git merge --no-ff feature/mi-feature

# Taggear una version
git tag -a v1.2.0 -m 'Release 1.2.0' && git push origin --tags

# Ver commits de una rama especifica
git log --oneline develop..feature/mi-feature
```